pid 1（user-image-1）：task_struct、cpu_context、pt_regs、内核栈、用户地址空间与 L0→L3 页表

move_to_user_mode 之后的快照；日志里的内核地址只打印了低 32 位

task_struct • PA 0x1000000（整页）

cpu_context（task_struct 的第一个字段，偏移 0） 0x1000000

x19	0x825e0	要运行的函数 fn
x20	0x0	fn 的参数
pc (x30)	0x8c934	ret_from_fork
sp	0x1001ef0	指向内核栈里的 pt_regs ⟶
x21–x29, fp	…	其余被调用者保存寄存器

state / counter	TASK_RUNNING / 时间片
mm.pgd	0x1005000 → 用户页表根（写入 TTBR0_EL1）
mm.pages[]	本进程分配的物理页（退出时释放）
…	task_struct 的其余字段

内核栈页 [0x1001000, 0x1002000) • PA 0x1001000

0x1002000 栈顶

pt_regs / trapframe（272 B = 0x110）

regs[0..30]	248 B	x0–x30（新进程为 0）
sp	8 B → SP_EL0	0x8000000
pc	8 B → ELR_EL1	0x400000
pstate	8 B → SPSR_EL1	0x3c0

0x1001ef0 = 栈顶 − 0x110

↓ 空闲内核栈（向低地址增长）

之后进入 EL1：kernel_entry 在同一位置保存 pt_regs；

被调度切走：cpu_context.sp 存成比 0x1001ef0 更低的值。

0x1001000 栈底

eret 时的系统寄存器（值来自 pt_regs）

TTBR0_EL1	0x1005000	用户页表（每进程不同）
TTBR1_EL1	0xa0000	内核页表（所有进程共享）
SP_EL0	0x8000000	用户栈顶
ELR_EL1	0x400000	eret 后的 PC
SPSR_EL1	0x3c0	EL0t，DAIF 全屏蔽

日志中这三行是“倒着印”的：

0000008000000000 → 0x8000000

0000040000000000 → 0x400000

0c30000000000000 → 0x3c0

（十六进制位的输出顺序反了）

TTBR0_EL1 = mm.pgd = 0x1005000（遍历起点）

PA 0x1000000 **task_struct** 含 cpu_context

PA 0x1001000 **内核栈** 顶部是 pt_regs

…… 其它页

PA 0x1005000 **L0 / PGD** [0] = 0x1006003 —[0]→

PA 0x1006000 **L1 / PUD** [0] = 0x1007003 —[0]→

L2 / PMD PA 0x1007000

- [2] = 0x1008003 → PA 0x1008000
- [63] = 0x100a003 → PA 0x100a000

L3 / PTE（两张表，各占一页）

- PA 0x1008000 [0] = 0x1009ec3
- PA 0x100a000 [511] = 0x100be43

物理页（地址从左到右递增）：箭头就是 MMU 的遍历——描述符的高位是下一级表/页的 PA；最后一级指向的两页画在下方

用户虚拟地址空间（TTBR0_EL1 = 0x1005000）

0x8000000 ← SP_EL0

用户栈页（RW）

VA [0x7fff000, 0x8000000)

L2=63 L3=511 → PA 0x100b000

0x7fff000

未映射 — 未映射（访问 → 缺页异常）

0x401000

用户映像（代码）页 只读

VA [0x400000, 0x401000)

首条指令 stp x29, x30, [sp, #-32]!

L2=2 L3=0 → PA 0x1009000

0x400000 ← ELR_EL1

0x0

映射到 → **PA 0x100b000** 物理页：用户栈页 属性 0xe43（RW）

PTE[511] → 页 0x100b000

映射到 → **PA 0x1009000** 物理页：代码页 属性 0xec3（只读）

PTE[0] → 页 0x1009000

用户栈 VA = 0x7fff000

47:39	38:30	29:21	20:12	11:0
L0 = 0 页表下标	L1 = 0 页表下标	L2 = 63 页表下标	L3 = 511 页表下标	偏移 = 0 页内偏移

代码 VA = 0x400000

47:39	38:30	29:21	20:12	11:0
L0 = 0 页表下标	L1 = 0 页表下标	L2 = 2 页表下标	L3 = 0 页表下标	偏移 = 0 页内偏移

这些下标就是物理页行箭头上标的 [0] [0] [63] [511] / [0] [0] [2] [0]；两条链在 L2/PMD 处分叉。

PTE 属性解码

字段	位	代码页 0xec3	栈页 0xe43
Valid / 页	1:0	11	11
AttrIndx	4:2	0 → MAIR.Attr0	0 → MAIR.Attr0
AP[2:1]	7:6	11 EL1/EL0 只读	01 EL1/EL0 读写
SH	9:8	10 外部共享	10 外部共享
AF	10	1 已访问	1
nG	11	1 非全局（ASID）	1
PXN / UXN	53 / 54	0 / 0	0 / 0 ⚠ 栈可执行

- 表描述符 0x…003：Valid=1、Table=1，高位是下一级表的 PA。
- 栈页通常应置 UXN=1；用户页通常应置 PXN=1（内核不执行用户代码）。
- AttrIndx=0 要确认 MAIR_EL1.Attr0 是 Normal 内存，而不是 Device。

这几样东西的关系

① **copy_process：分配 task_struct（0x1000000）和内核栈（0x1001000）；**

在栈顶留出 pt_regs（0x1001ef0），cpu_context.sp 指向它，pc = ret_from_fork，x19 = fn。

② **第一次被调度：cpu_switch_to 恢复 x19–x30 和 sp，ret → ret_from_fork → 调用 fn。**

③ **fn 里 move_to_user_mode：分配页表 5 张 + 代码页 + 栈页（0x1005000–0x100b000），**

把用户程序复制进代码页，填写 pt_regs：pc=0x400000、sp=0x8000000、pstate=0x3c0，

并把 mm.pgd = 0x1005000 写进 TTBR0_EL1。

④ **返回 ret_from_fork 后 kernel_exit：从 pt_regs 恢复寄存器，eret 进入 EL0。**

⑤ **之后每次系统调用/中断：硬件切到 SP_EL1 = 本进程内核栈，**

在同一位置 0x1001ef0 重新保存 pt_regs；用户态 sp 存在其中的 sp 字段。

⚠ **pstate 0x3c0 屏蔽 IRQ：用户态不会被定时器抢占，Linux 用 0x0。**

⚠ **内核只打印了 2 MB 块映射（PA 0–2 MB），0x1000000 处的 task_struct**

必然还有别的映射；TTBR1 walk 用的 L0[511] 也值得和内核基址核对。